

# CheckInOut – Hostel Management System

CS 432 – Databases (Course Project – Assignment 3)

Module A: B+ Tree DBMS: Transaction Management & ACID Validation

Semester II (2025–2026)

Date: 05 April 2026

Instructor: Dr. Yogesh K. Meena

## Team Members & Contributions

Vipul Sunil Patil – WAL implementation with LSN ordering, JSON logging, fsync durability, and thread-safe appends.

Sai Krishna – Report writing, formatting, diagrams, experimental analysis, and video/demo coordination.

Daksh Jain – Testing framework with 15 ACID test cases, stress testing, concurrency and failure simulation.

Daksh Dave – Transaction Manager design (begin, commit, rollback), buffering, atomic apply/undo logic, and isolation integration with DBManager

Harshit – Recovery Manager using ARIES (Analysis, Redo, Undo), crash simulation, and WAL parsing.

## GitHub Repository:

<https://github.com/vipulSP2108/CheckInOut-Hostel-Management-System>

## Video Link:

<https://youtu.be/qOfLjtESC3E>

## Abstract

This report documents the design, implementation, and experimental validation of Module A for CS 432 Assignment 3. We extend the Assignment 2 B+ Tree DBMS with a complete transaction engine that provides all four ACID guarantees—Atomicity, Consistency, Isolation, and Durability—together with ARIES-inspired crash recovery via a Write-Ahead Log (WAL). Three new modules are added entirely above the existing engine without modifying any Assignment 2 code. A 15-test automated suite verifies every property. All 15 tests pass, and simulated crash-recovery experiments confirm that the committed database state is rebuilt correctly from the WAL alone, with no partial data remaining after a mid-transaction failure.

## Keywords

ACID, Write-Ahead Log, B+ Tree, Transaction Management, Crash Recovery, ARIES, Isolation, Atomicity, Durability

## 1 Introduction & Operating Principles

The Assignment 2 DBMS stored records in an in-memory B+ Tree and exposed CRUD operations through a DatabaseManager. While functionally correct under single-user, no-failure conditions, it offered no safety guarantees: a crash mid-insert could leave a half-written index, two concurrent inserts of the same primary key could both succeed, and committed data vanished on process exit. Module A resolves all three problems.

Our implementation is guided by the four ACID properties:

- **Atomicity.** Every transaction either applies in full or has no effect at all.
- **Consistency.** Primary-key constraints and DB / B+ Tree synchronisation hold before and after every transaction.
- **Isolation.** Each transaction sees only committed data from other transactions (READ COMMITTED level).
- **Durability.** Once `commit()` returns, the data survives a crash. Committed rows are rebuilt from the WAL on restart.

### 1.1 System Context

The three new modules sit in the database/ package alongside the existing files. The existing files—`bplustree.py`, `table.py`, `db_manager.py` are untouched.

Table 1: New Module Responsibilities

| File                   | Responsibility                                                                                    |
|------------------------|---------------------------------------------------------------------------------------------------|
| wal.py                 | Append-only fsync'd JSON-line log; stores before/after images; monotone LSN ordering; thread-safe |
| transaction_manager.py | begin / commit / rollback; op buffering; READ COMMITTED isolation                                 |
| recovery.py            | 3-pass ARIES recovery: Analysis → Redo → Undo                                                     |
| tests/test_acid.py     | 15 automated tests covering all 4 ACID properties                                                 |

## 2 Architectural Foundation & Tooling

### 2.1 Core Components

*wal.py*. Every mutation is written to the WAL *before* it touches the B+ Tree. Each record is one JSON line:

```
1 {"lsn": 2, "txn_id": "T-A1B2",
2  "op": "INSERT", "table": "students",
3  "key": 1,
4  "before": null,
5  "after": {"id":1,"name":"Alice","gpa":9.1},
6  "ts": "2026-04-04T10:00:00"}
```

Listing 1: WAL record format (wal.py)

The `lsn` (Log Sequence Number) is a monotonically increasing integer that globally orders every record across all concurrent transactions. `before` stores the row state *before* the mutation; `after` stores the state *after*. Both images are required for undo and redo during recovery.

`os.fsync()` is called after every append to flush the OS write buffer to physical storage; the critical step that makes durability real. A `threading.Lock` serialises concurrent appends.

*transaction\_manager.py*. Provides the `TransactionManager` class. A second `threading.Lock` guards the active-transaction registry (`_txns`). Every DML call buffers a `PendingOp` in memory; the B+ Tree is never touched until `commit()` is called.

*recovery.py*. Implements the three-pass ARIES algorithm over the WAL file. Called once at startup; safe to call multiple times (idempotent redo).

### 2.2 Transaction Lifecycle

```
1 # Step 1 -- begin
2 txn_id = tm.begin()
3 # WAL: { op: BEGIN, txn_id }
4 # Creates Transaction with empty op buffer
5
6 # Step 2 -- buffer (B+ Tree untouched)
7 tm.insert(txn_id, "students", row)
8 # WAL: { op: INSERT, key, after=row }
9 # PendingOp appended to in-memory buffer
10
11 # Step 3a -- commit (all-or-nothing apply)
12 tm.commit(txn_id)
13 # _apply_ops() -> table.insert_row()
14 # -> BPlusTree.insert()
15 # WAL: { op: COMMIT } <-- fsynced here
16
17 # Step 3b -- rollback (zero ops reach DB)
18 tm.rollback(txn_id)
19 # WAL: { op: ABORT }
20 # Buffer discarded; B+ Tree untouched
```

Listing 2: Transaction data flow

### 2.3 Endpoint Priority & Rationale

By analogy with the CheckInOut deployment model, we identify the analogous critical operations in the DBMS engine:

Table 2: Operation Priority Matrix

| Priority | Operation | Rationale                                  |
|----------|-----------|--------------------------------------------|
| CRITICAL | Commit    | All-or-nothing; must be atomic and durable |
| CRITICAL | Rollback  | Must leave zero side-effects               |
| HIGHEST  | WAL fsync | Without this, durability is illusory       |
| HIGH     | Recovery  | System must self-heal after crash          |
| MEDIUM   | Select    | Read-committed snapshot; no dirty reads    |

### 3 Correctness of Operations

#### 3.1 Atomicity: The Buffer Pattern

Operations are buffered in a list[PendingOp] inside the Transaction object. The DatabaseManager is untouched until `_apply_ops()` is called during `commit()`.

```

1 def _apply_ops(self, ops):
2     applied = []
3     for op in ops:
4         tbl = self._db.get_table(op.table)
5         if op.op == "INSERT":
6             tbl.insert_row(op.after)
7         elif op.op == "UPDATE":
8             tbl.update_row(op.key, op.after)
9         elif op.op == "DELETE":
10            tbl.delete_row(op.key)
11        applied.append(op)
12
13 def _undo_partial(self, ops):
14     # Reverse already-applied ops on mid-commit failure
15     for op in reversed(ops):
16         tbl = self._db.get_table(op.table)
17         if op.op == "INSERT":
18             tbl.delete_row(op.key)
19         elif op.op == "UPDATE":
20             tbl.update_row(op.key, op.before)
21         elif op.op == "DELETE":
22             tbl.insert_row(op.before)

```

Listing 3: `_apply_ops` and `_undo_partial` (transaction\_manager.py)

If any op throws during `_apply_ops()`, the exception is caught, `_undo_partial()` reverses what was already applied in reverse order, and ABORT is written to the WAL. The database is left exactly as it was before `commit()` was called.

#### 3.2 Consistency: DB and B+ Tree Synchronisation

The Table class is the single write path: every `insert_row()`, `update_row()`, and `delete_row()` call updates both the relational row store and the B+ Tree index atomically. Since the transaction engine only calls Table methods (never the B+ Tree directly), the two layers can never diverge.

Tests T05 and T06 verify this explicitly:

```

1 db_rows = db.select_all("students")
2 tree_rows = [v for _, v in
3               tbl._index.get_all()]
4
5 self.assertEqual(len(db_rows), len(tree_rows))
6 for db_r, tree_r in zip(db_rows, tree_rows):
7     self.assertEqual(db_r["id"], tree_r["id"])

```

Listing 4: DB vs B+ Tree consistency check (test\_acid.py)

#### 3.3 Isolation: READ COMMITTED

The `_txn_visible_select()` gate implements READ COMMITTED isolation. It reads the committed state from the DatabaseManager and then overlays *only the current transaction's* own buffered ops:

```

1 def _txn_visible_select(self, txn, table, pk):
2     # 1. Committed state only (other txns invisible)
3     row = self._db.select(table, pk)
4     row = dict(row) if row else None
5
6     # 2. Overlay this txn's own pending ops
7     for op in txn.ops:
8         if op.table != table or op.key != pk:
9             continue
10        if op.op == "INSERT": row = dict(op.after)
11        elif op.op == "UPDATE": row = dict(op.after)
12        elif op.op == "DELETE": row = None
13    return row # never contains another txn's pending data

```

Listing 5: Isolation gate (transaction\_manager.py)

This prevents dirty reads: if T\_A inserts row 42 but has not committed, T\_B's `select(42)` returns None because (1) the DatabaseManager has no row 42 yet, and (2) T\_B has no such op in its own buffer.

#### 3.4 Durability: WAL-Before-DB Rule

**Rule:** Every WAL record is fsynced to disk *before* its corresponding mutation reaches the B+ Tree. The COMMIT record is the last WAL write in `commit()`. If `commit()` returns without an exception, the COMMIT record is guaranteed on disk. A crash immediately after cannot lose it.

```

1 def append(self, txn_id, op, ...):
2     with self._lock:
3         self._lsn += 1
4         rec = WALRecord(self._lsn, txn_id, op, ...)
5         line = json.dumps(rec.to_dict()) + "\n"
6         with open(self._path, "a") as f:
7             f.write(line)
8             f.flush()
9             os.fsync(f.fileno()) # <- durability guarantee
10    return rec

```

Listing 6: fsync on every WAL append (wal.py)

### 4 Failure Handling & Crash Recovery

#### 4.1 The Challenge: Mid-Transaction Crash

Without a WAL, a crash between “insert row A” and “insert row B” in the same transaction leaves the database in an inconsistent state that cannot be detected or fixed on restart. Our WAL records every intention before execution, making all states recoverable.

#### 4.2 WAL Record Fields

#### 4.3 Three-Pass ARIES Recovery

On startup, `RecoveryManager.recover()` runs three passes:

#### 4.4 Crash Scenario Walk-Through

The following is the exact scenario from Test T12:

```

1 WAL on disk after crash:
2 LSN 1 T-OK BEGIN
3 LSN 2 T-OK INSERT key=10 after={id:10,...}
4 LSN 3 T-OK INSERT key=11 after={id:11,...}
5 LSN 4 T-OK COMMIT <- finished cleanly
6 LSN 5 T-CRASH BEGIN
7 LSN 6 T-CRASH INSERT key=99 after={id:99,...}
8 <- power cut here. No COMMIT for T-CRASH.
9
10 Recovery:

```

Table 3: WAL Record Schema

| Field      | Type          | Purpose                                                                 |
|------------|---------------|-------------------------------------------------------------------------|
| lsn        | int           | Global ordering; recovery uses LSN sort                                 |
| txn_id     | str           | UUID identifying the owning transaction                                 |
| op         | str           | BEGIN INSERT UPDATE DELETE COMMIT ABORT                                 |
| table      | str None      | Target table (null for lifecycle records)                               |
| key before | any dict None | Primary key of the affected row<br>Pre-mutation row (required for UNDO) |
| after      | dict None     | Post-mutation row (required for REDO)                                   |
| ts         | str           | ISO timestamp for audit trail                                           |

Table 4: ARIES Recovery Passes

| Pass        | Action                                                                                                                                      |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Analysis | Scan entire WAL. Classify each transaction: COMMITTED, ABORTED, or ACTIVE (loser – was in-flight at crash).                                 |
| 2. Redo     | Re-apply every INSERT/UPDATE/DELETE from COMMITTED transactions in LSN order. Idempotent: checks current state before applying.             |
| 3. Undo     | For each loser transaction, walk its records in <i>reverse</i> LSN order. Apply inverse operations using before-images. Write ABORT to WAL. |

```

11 Analysis : T=OK=COMMITTED, T=CRASH=LOSER
12 Redo    : insert row 10, insert row 11 (T=OK)
13 Undo    : delete row 99 (T=CRASH)
14 WAL write: ABORT T=CRASH
15
16 Final DB: rows {10, 11} present. Row 99 absent.

```

Listing 7: WAL state at crash + recovery result

## 4.5 Failure Simulation Code

```

1 # Manually append partial WAL -- no COMMIT
2 wal_raw.append("T-CRASH", "BEGIN")
3 wal_raw.append("T-CRASH", "INSERT",
4               "students", 99,
5               before=None,
6               after=make_student(99))
7 # <- no COMMIT written (simulates power cut)
8
9 # Recovery on a fresh empty DB
10 db2 = fresh_db("T12_s2")
11 rm = RecoveryManager(db2, wal_path=wal_path)
12 report = rm.recover()
13
14 assert "T-CRASH" in report.loser_txns
15 assert db2.count("students") == 1
16 assert db2.select("students", 10) is not None
17 assert db2.select("students", 99) is None

```

Listing 8: Simulated crash in Test T12 (test<sub>a</sub>cid.py)

## 4.6 Mid-Commit Failure Handling

If an exception is raised *during* `_apply_ops()` – for example, a duplicate PK on the third of five inserts – the method calls `_undo_partial()` immediately. It reverses in reverse order every op that already reached the B+ Tree, restoring the pre-commit state. Then ABORT is written to the WAL.

```

1 def commit(self, txn_id):
2     txn = self._get_active(txn_id)
3     try:
4         self._apply_ops(txn.ops)
5     except Exception as exc:
6         # Reverse partially applied ops
7         self._undo_partial(txn.ops)
8         self._wal.append(txn_id, OP_ABORT)
9         txn.state = TxnState.ABORTED
10        raise RuntimeError(
11            f"Commit {txn_id} failed -- auto-rolled back."
12        ) from exc
13    # All ops applied; now make durable
14    self._wal.append(txn_id, OP_COMMIT)
15    txn.state = TxnState.COMMITTED

```

Listing 9: Commit with cascade rollback (transaction\_manager.py)

## 5 Multi-User Conflict Handling

### 5.1 Thread-Safety Design

Two locks protect shared state:

- **WAL lock** (`threading.Lock` in `WALManager`): serialises file appends so that concurrent transactions never interleave bytes within a single JSON line.
- **Registry lock** (`threading.Lock` in `TransactionManager`): guards the `_txns` dict during `begin()`, `commit()`, and `rollback()`.

The two locks are independent and never held simultaneously, eliminating deadlock between them.

### 5.2 Dirty Read Prevention

Each Transaction's `_ops` list is a private in-memory object. No other transaction can access it. The only shared mutable state is the committed `DatabaseManager`. Therefore:

- (1) T\_A inserts row 42. Not yet committed.
- (2) T\_B calls `tm.select(t_b, "students", 42)`.
- (3) `_txn_visible_select` reads `DatabaseManager` (row 42 absent), then overlays T\_B's own buffer (also no row 42).
- (4) Returns `None`. Dirty read impossible.

### 5.3 Race Condition Scenario: Concurrent PK Insert

If two transactions both attempt to insert the same primary key:

```

1 T:0ms T_A: tm.insert(t_a, "students",
2               {id:5, name:"Alice"})
3         # Check committed DB: id=5 absent -> OK
4         # Buffer PendingOp(INSERT, key=5)
5
6 T:1ms T_B: tm.insert(t_b, "students",
7               {id:5, name:"Bob"})
8         # Check committed DB: id=5 still absent
9         # (T_A not committed yet)
10        # Buffer PendingOp(INSERT, key=5)
11
12 T:2ms T_A: tm.commit(t_a)
13        # _apply_ops: table.insert_row({id:5}) -> OK
14        # WAL: COMMIT T_A
15
16 T:3ms T_B: tm.commit(t_b)

```

```

17 # _apply_ops: table.insert_row({id:5})
18 # -> ValueError: Duplicate PK
19 # _undo_partial: nothing to undo (T_B's apply failed)
20 # WAL: ABORT T_B
21 # RESULT: T_A committed; T_B rolled back cleanly

```

**Listing 10: Race timeline – concurrent PK conflict**

The PK duplicate is caught at apply time, not at buffer time, when two transactions independently read a clean committed state. The second committer loses cleanly with a `RuntimeError`. No data corruption occurs.

## 5.4 Concurrent Transactions on Different Tables (T14)

Test T14 launches two threads simultaneously. Thread 1 commits 5 student rows; Thread 2 commits 3 course rows. Because the two transactions touch different tables and the WAL lock only serialises file writes (not the entire commit), both complete successfully.

```

1 def insert_students():
2     t = tm.begin()
3     for i in range(1, 6):
4         tm.insert(t, "students", make_student(i))
5     tm.commit(t)
6
7 def insert_courses():
8     t = tm.begin()
9     for c in range(1, 4):
10        tm.insert(t, "courses",
11                {"cid": c, "title": f"CS-{c*100}",
12                 "credits": 3})
13    tm.commit(t)
14
15 th1 = threading.Thread(target=insert_students)
16 th2 = threading.Thread(target=insert_courses)
17 th1.start(); th2.start()
18 th1.join(); th2.join()
19
20 assert db.count("students") == 5
21 assert db.count("courses") == 3

```

**Listing 11: Concurrent thread test (test\_acid.py T14)**

## 6 Stress Testing and Performance Analysis

### 6.1 Experimental Design

All stress tests run within the Python `unittest` framework without external tools. Test T15 provides the primary load scenario: 200 rows inserted in 4 batches of 50, each batch a separate committed transaction. Additional experiments target isolation and recovery under simulated crash conditions.

### 6.2 Stress Test Scenarios

**Table 5: Stress Test Scenarios and Parameters**

| Scenario           | Rows | Txns | Objective                    |
|--------------------|------|------|------------------------------|
| Sequential inserts | 200  | 4    | B+ Tree balance under load   |
| Rollback storm     | 200  | 100  | Zero leakage on mass abort   |
| Crash mid-txn      | 3    | 2    | ARIES recovery correctness   |
| PK race (serial)   | 1    | 2    | Conflict resolution          |
| Concurrent threads | 8    | 2    | Thread-safety on diff tables |
| WAL read back      | 200  | 4    | Durability after “shutdown”  |

### 6.3 Stress Test: 200 Sequential Inserts (T15)

```

1 BATCH = 50
2 for batch_start in range(0, 200, BATCH):
3     t = tm.begin()
4     for i in range(batch_start, batch_start + BATCH):
5         tm.insert(t, "students", make_student(i))
6     tm.commit(t)
7
8 self.assertEqual(db.count("students"), 200)
9 # Spot checks at boundary keys
10 for pk in [0, 50, 100, 150, 199]:
11     row = db.select("students", pk)
12     self.assertIsNotNone(row)
13     self.assertEqual(row["id"], pk)

```

**Listing 12: Stress test T15 – 200 rows in 4 batches**

With order-4 B+ Tree, inserting 200 keys causes approximately  $\log_4 200 \approx 4$  levels of height and requires  $\approx 35$  node splits across all batches. All splits maintain correct leaf-linked-list pointers, verified by `select_all()` returning exactly 200 rows in sorted order.

### 6.4 Gate-Scan Analogy: WAL Concurrency

Analogous to CheckInOut’s gate-scan optimisation (where BEGIN IMMEDIATE was removed to allow parallel fire-and-forget inserts), our B+ Tree WAL does not require transaction locking for simple read operations. Only mutations go through the transaction buffer; `select()` bypasses it and reads directly from the committed DatabaseManager. This mirrors the WAL-mode read/write decoupling observed in CheckInOut’s optimised gate endpoint.

## 7 Experimental Results & Integrity Validation

### 7.1 Test Suite Summary

All 15 ACID tests pass. Total execution time:  $\approx 0.76$  seconds on CPython 3.12.3.

### 7.2 Verification Logs

**Table 6: Experiment Outcomes and Verification Logs**

| Experiment             | Size        | Result                | Verified |
|------------------------|-------------|-----------------------|----------|
| Commit atomicity       | 3 inserts   | All 3 appear          | PASS     |
| Rollback atomicity     | 2 inserts   | 0 rows written        | PASS     |
| PK uniqueness          | 2 txns      | 2nd rejected          | PASS     |
| DB == B+ Tree          | 8 rows      | Lists identical       | PASS     |
| Dirty-read             | T_A, T_B    | T_B sees None         | PASS     |
| Concurrent txns        | T_A, T_B    | Neither sees other    | PASS     |
| WAL COMMIT record      | 1 commit    | Record on disk        | PASS     |
| Recovery rebuild       | 3 committed | All 3 restored        | PASS     |
| Recovery ignores abort | 1 aborted   | Absent after recovery | PASS     |
| Crash mid-txn          | 1 crash     | Loser undone          | PASS     |
| Cascade rollback       | 1 mid-apply | DB unchanged          | PASS     |
| Concurrent threads     | 5+3 rows    | Both correct          | PASS     |
| Stress 200 inserts     | 200 rows    | count==200            | PASS     |

### 7.3 Data-Driven Observations

*Atomicity.* The before-commit read confirms the B+ Tree is never written until `commit()` is called. The rollback test confirms zero side-effects. All 4 atomicity tests (T01, T02, T03, T13) pass.

*Consistency.* Tests T05 and T06 walk both `db.select_all()` and `bplustree.get_all()` independently and compare key lists. They are identical in all scenarios tested.

*Isolation.* Test T07 demonstrates the dirty-read gate: T\_A's uncommitted insert is invisible to T\_B. Test T08 confirms two concurrent transactions with no cross-visibility of pending writes. Test T14 verifies thread-safe concurrent commits on different tables.

*Durability and Recovery.* Test T10 simulates a "restart": three rows committed in Session 1, a fresh empty DB created as Session 2, recovery run – all three rows rebuilt. Test T12 goes further: a crash is simulated by writing a partial WAL manually. The loser transaction is undone, and the committed rows survive.

*B+ Tree height under load.* After inserting 200 keys with order-4, the tree height is  $\lceil \log_4 200 \rceil = 4$  levels. `select_all()` traverses the leaf linked list in  $O(n)$  and returns all 200 rows in sorted order, confirming correct split maintenance throughout.

### 7.4 Overall Observations

The most significant insight is the importance of the **WAL-before-DB rule**. Writing and fsyncing each mutation before it touches the B+ Tree is the single design decision that makes all four ACID properties achievable. Without it, recovery would require scanning the entire index to detect inconsistencies.

The before-image stored with every UPDATE and DELETE record makes the Undo pass straightforward: no separate undo log is required. Storing both images in the same WAL record is more space-efficient than the classic separate undo-log design and simplifies the recovery code.

## 8 Ensuring Operations Correctness

The framework ensures correctness across all ACID pillars:

- **Atomicity.** The buffer pattern guarantees all-or-nothing: either `_apply_ops()` completes without exception, or `_undo_partial()` reverses every partially applied op.
- **Consistency.** PK uniqueness is checked against the committed state before buffering any insert. DB/B+ Tree synchronisation is verified structurally by tests T05 and T06.
- **Isolation.** `_txn_visible_select()` acts as an isolation gate: reads from committed state only, never from another transaction's buffer. Verified by T07, T08, and T14.
- **Durability.** `os.fsync()` on every WAL append. COMMIT record written last. Three-pass ARIES recovery rebuilds any committed state from the WAL alone. Verified by T09, T10, T11, T12.

## 9 Limitations & Scalability

- **In-memory only.** The B+ Tree and DatabaseManager are in RAM. After a clean shutdown, all committed data is lost unless the table schemas are re-created before recovery is run. The WAL alone cannot recreate the schema.

- **No row-level locking.** Two transactions updating the same row will not deadlock (buffers are independent) but the second committer loses silently (`RuntimeError`). Optimistic Concurrency Control (OCC) or Two-Phase Locking (2PL) is not implemented.
- **Single WAL writer.** A single mutex serialises all WAL appends. Under very high concurrency, this becomes a bottleneck, analogous to CheckInOut's BEGIN IMMEDIATE over-locking issue on gate scans. Group commit would resolve this.
- **No automatic checkpointing.** The WAL grows unboundedly. A long-running process must call `checkpoint()` manually. Without it, recovery time grows linearly with WAL size.
- **READ COMMITTED only.** Phantom reads and non-repeatable reads are possible. SERIALIZABLE isolation requires predicate locks or MVCC.
- **No nested transactions.** SAVEPOINTS are not supported. Each `txn_id` is either fully committed or fully rolled back.
- **Python GIL.** The GIL limits true parallelism; Test T14 demonstrates correct isolation under threading but not true simultaneous CPU execution. A multi-process or async design would be needed for production throughput.

## 10 Conclusion

Through the surgical addition of three modules above the Assignment 2 B+ Tree DBMS, we have successfully implemented a complete transaction engine with full ACID guarantees. The Write-Ahead Log provides durability and the data needed for crash recovery. The Transaction Manager provides atomicity and READ COMMITTED isolation via an in-memory op buffer. The Recovery Manager implements ARIES-style three-pass recovery that handles both committed-state redo and loser-transaction undo.

Our 15-test automated suite verifies every guarantee and all six evaluation criteria. All 15 tests pass. Crash-recovery experiments confirm that the database is rebuilt to exactly the last consistent committed state from the WAL alone, with no partial or ghost records remaining.

The key architectural insight – analogous to CheckInOut's WAL-mode read/write decoupling – is the **WAL-before-DB rule**: every mutation must be logged and fsynced before it touches the B+ Tree. This single rule is what makes all four ACID properties achievable at low implementation cost.